



SOFTWARE QUALITY REPORT

Collabify against ISO/IEC 25010:2023

What was measured, what was found, what was changed, and what is still open. Every figure in this document was taken from the running system; anything that could not be measured here says so.

SYSTEM

Collabify — coursework workspace for the BSIT program, Dr. Yanga's Colleges, Inc.

STACK

React 19 · TypeScript · Vite 7 · Tailwind v4 · Supabase (PostgreSQL with row-level security)

PREPARED

23 August 2026

SCOPE

Nine quality characteristics of ISO/IEC 25010:2023

What this document is, and is not

ISO/IEC 25010 is a **quality model**. It names nine characteristics that software can be described against. It is not a certification, there is no certificate to hold, and no software is ever "ISO 25010 certified" — anyone claiming otherwise is describing something else. This document uses the model the way it is meant to be used: as a checklist to be honest against.

The work behind it ran in phases, riskiest first. Each phase measured the system before changing it, changed it, and measured it again. The numbers in this report are those measurements, not estimates.

How to read the verdicts

Addressed Measured before, changed, measured after. The figures are here.

Partial Improved and measured, with a named part still outstanding.

Open Examined and deliberately not changed. The reason is given.

The system, in figures

Database size	17 MB
Tables in public	36
Views	35
Row-level security policies	77
Triggers	52
Database test suites	15
Passing assertions	379
Failing assertions	0

```
REPEAT EVERY DATABASE MEASUREMENT
```

```
for f in supabase/tests/*.test.sql; do node scripts/db.mjs "$f"; done
```

Defects found, in one table

The strongest evidence that an exercise like this was worth doing is what it turned up. Eighteen defects were found. Five of them were affecting people using the live system; the rest were latent.

#	DEFECT	CHARACTERISTIC	WAS LIVE
1	Deleting one professor's account destroyed their class, its projects, boards, tasks, files and enrolments — no warning, no undo	Safety	latent
2	No error boundary: one thrown error blanked the whole application with no way back	Reliability	live
3	Eight pages showed a fetch error with no way to retry but a manual reload	Reliability	live
4	No signal when the connection dropped — writes failed silently on campus wifi	Reliability	live
5	<code>guard_task_edit</code> pinned <code>created_by</code> back, so a professor could not be deleted even after handing the class over	Safety	latent
6	The account page promised an approval email that is never sent	Functional suitability	live
7	Whole application shipped as one 1.15 MB file — the landing page carried the admin console	Performance	live
8	School logo shipped at 2048×2048, 1.7 MB, for a 56-pixel slot	Performance	live
9	Analytics views re-ran correlated subqueries per row; cost grew as members × boards	Performance	live
10	<code>task_member_progress</code> lost <code>cap_pct</code> and <code>can_claim</code> , so the assignee picker offered a claim the database refuses — a student met an exception instead of a disabled option	Functional suitability	live
11	<code>task_board_overview</code> missing ten columns in the file that runs last — rebuilding from the documented order produced a database the app could not query	Maintainability	latent
12	<code>guard_task_assignee</code> lost the locked-project and handed-in checks to an older copy in a later file	Maintainability	latent
13	<code>guard_reassignment_request</code> lost the handed-in check the same way	Maintainability	latent
14	Control borders at 1.95:1 (light) and 2.32:1 (dark) — WCAG 1.4.11 requires 3:1	Interaction	live
15	<code>focus:outline-none</code> on inputs removed the only focus indicator meeting 3:1	Interaction	live

#	DEFECT	CHARACTERISTIC	WAS LIVE
16	No focus trap: Tab walked out of every dialog into the page hidden behind it	Interaction	live
17	Landing navbar pushed the menu icon off a 360 px screen, where it could not be tapped	Flexibility	live
18	CSV exported timestamps as raw UTC <code>timestamp_{tz}</code> , which Excel stores as unsortable text	Compatibility	live

All eighteen are fixed. Numbers 11 to 13 share one cause and are worth singling out: several files define the same view or trigger function, and whichever runs *last* wins. Three times that was the older copy, silently undoing a newer one. `scripts/schema-drift.mjs` now reports every such object and flags any whose last definition is smaller than an earlier one.

Functional suitability

Completeness, correctness and appropriateness — does it do the tasks it is meant to, and get them right?

The feature set is complete for what the product claims: classes and syllabus weeks, group sets, projects on boards, fair-share task claiming, submission and a professor's answer, four-layer analytics, reports for both professor and student, and a program office console.

What was wrong

Two places where the interface and the database disagreed, which is the failure mode that matters most here — the user is told one thing and the system does another.

- **The assignee picker offered claims the database refuses.** `task_member_progress` had lost `cap_pct` and `can_claim` to an older redefinition, while three components still read them. `can_claim ?? true` resolved to *true* for everyone, so the fair-share ceiling was invisible and the "share is full" hint could never appear. The rule itself never lapsed — `guard_task_assignee` still raises — so a student was refused by a database exception rather than by a greyed-out option.
- **The pending-approval page promised an email** that is never sent, because outbound mail is not switched on. It now tells the professor to sign in again to check.

Addressed. Both columns restored and asserted; the copy no longer promises what the system cannot do.

VERIFY

```
node scripts/db.mjs supabase/tests/deadline-lock.test.sql
```

24 assertions, including the claim ceiling.

Performance efficiency

Time behaviour, resource use and capacity, relative to what the system is asked to do.

What a first-time visitor downloads

	BEFORE	AFTER
JavaScript, as built	1,150,275 B in 1 file	split across 74 chunks
First visit, on the wire	~333 KB	219 KB
Files fetched on first visit	2	5
School logo	1,718,585 B	39,757 B
dist/ total	2,953,103 B	1,314,763 B

Thirty-three routes are loaded on demand, split by what changes together rather than by size, so a deploy that changes one line of copy no longer makes every returning student re-download React, Supabase and the animation library. **82,728 bytes of admin and analytics code are never sent to a student at all.**

The logo was larger than the entire application. It was resized by `scripts/resize-png.mjs`, written for the purpose rather than adding an image library for one file, and its output was checked against the browser's own canvas downscale: mean channel difference 1.12 of 255, worst case 3.

Query time, against the live database

VIEW	BEFORE	AFTER
<code>class_actions</code> — the prescriptive analytics band	421.8 ms	116 ms
<code>class_participation</code>	133.2 ms	8.9 ms
<code>board_diagnosis</code>	91.9 ms	4.5 ms
<code>report_student_work</code>	58.2 ms	30.9 ms

`class_actions` is a median of five consecutive runs; 115–158 ms was observed, the high end on a cold plan.

The cause was not missing indexes — those were already right, and 591 rows is not enough data to be slow. `explain analyze` showed correlated lateral joins being re-executed per row: `class_participation` re-scanned every board once per student (320 loops), `board_diagnosis` had four laterals doing the same per board, and `task_member_progress` ran a scalar subquery per task — 1,640 index scans, 517 ms. All are grouped joins now.

The shape mattered more than the milliseconds. The cost was growing as members × boards, which is unremarkable at 591 rows and would not have been at a full cohort's.

One rewrite measured *worse* and was reverted: `distinct on` with a window count would not hash-join and ran 78 ms × 20 boards inside `class_actions`. A plain `GroupAggregate` does hash-join. The reason is recorded in the file so it is not attempted again.

Addressed.

Compatibility

Co-existence — sharing an environment without harming other software — and interoperability, exchanging information other systems can use.

Interoperability: the CSV export

Reports export to CSV for Excel and Google Sheets. Verified against the real function in a browser:

UTF-8 byte-order mark	present — Excel shows <i>Peña</i> , not <i>PeÃ±a</i>
Line endings	CRLF, per RFC 4180
Quotes inside a field	doubled
Commas and newlines in a field	field quoted
Empty value	empty cell, not the word <code>null</code>
Timestamps	2026-08-23 12:12, local time

The timestamp was a defect. `submitted_at` went into the file as the raw database value, 2026-08-23T04:12:33.123456+00:00. Excel and Sheets both store that as text, so the column could not be sorted or filtered as a date, and it read UTC rather than the Manila time the work was actually handed in. It is now written in the one format both parse unprompted, and which no regional setting can read backwards.

Print

Reports are meant to reach the program office on paper. The print stylesheet sets A4, forces the light palette, removes the rail, header and every control, flattens the scroll containers that fight pagination, repeats table headers across pages and keeps rows from breaking. No report sheet uses a coloured fill, so nothing depends on the reader having "background graphics" switched on — a common way for a status pill to print as white text on white paper.

Co-existence

Nothing is written to window. All four stored keys are namespaced — `collabify.theme`, `collabify.firstrun`, `collabify.nav.shut`, `collabify.sidebar.collapsed` — so the app cannot collide with anything else on the same origin.

Partial. An actual print preview needs a signed-in professor and has not been run; see the outstanding items.

Interaction capability

Recognisability, learnability, operability, user error protection, engagement, inclusivity and user assistance.

Contrast, computed rather than judged

`scripts/contrast.mjs` reads the design tokens straight out of the stylesheet and computes every text and control pair in both themes. It found three genuine failures:

PAIR	BEFORE	REQUIRED	AFTER
Faint text on the page (light)	4.34:1	4.5	4.54:1
Control border on a card (light)	1.95:1	3.0	3.20:1
Control border on a card (dark)	2.32:1	3.0	3.17:1

Input fields took their resting border from the decorative hairline token, so the fields were legible but their edges were not — which is the whole of what WCAG 1.4.11 is about. A dedicated `--control-line` token was solved for the worst background a field actually sits on, not for white. **Now 0 failing pairs in both themes.**

Keyboard

- **Focus trap and focus restore** in dialogs, the filter popover and the mobile navigation drawer. Tab used to walk out of a dialog and into the page behind it — invisible to a mouse user because the backdrop covers it, but on a keyboard you end up typing into a form you cannot see. Closing now hands focus back to the control that opened it, so the next Tab continues instead of restarting at the top.
- **Two full-screen backdrops were focusable buttons** labelled "Close", so the first thing announced inside a dialog was a control covering the entire viewport. They are inert now; the header has a real close button and Escape works.
- **focus:outline-none was removed** from inputs, selects and a task card. It had deleted the only indicator meeting 3:1, leaving a ring at 12% opacity that does not.
- **A skip link is the first tab stop** in the application shell, on the landing page and in the auth layout, each landing on a real focusable `#main-content`. The navigation rail is thirty-odd links, and without it a keyboard user walked all of them again on every page.
- **Escape closes the navigation drawer**, as it already did everywhere else.

User assistance

A dismissible first-run panel gives each role the three things it needs to know. An empty dashboard reads as broken rather than new — every panel says "nothing yet" — so it explains what will fill it and what to do first, then never appears again.

Addressed.

```
REPEAT
node scripts/contrast.mjs
node scripts/ally-names.mjs
```

0 failing pairs; 0 icon-only buttons without an accessible name.

Reliability

Faultlessness, availability, fault tolerance and recoverability.

- **An error boundary in two places.** Around the whole application, and — the one that matters — inside the shell around the page content, so a page that throws leaves the rail, the header and the way out alive. It says what failed in one sentence, offers *Try again* and a link to the dashboard, and logs the real error for whoever is debugging. It is keyed on the path, so navigating away from a broken page clears it.
- **Retry on failure.** Eight pages caught their fetch errors and displayed them with no way forward but a manual reload. The alert component now takes an `onRetry` and shows a button.
- **An offline bar** driven by the browser's own connection events, saying plainly that changes will fail until the connection returns. Campus wifi is the setting this runs in.
- **Recoverability is code.** The whole schema rebuilds from files, and that claim is now true — see Maintainability.

Addressed.

Security

Confidentiality, integrity, non-repudiation, accountability and authenticity.

This was the strongest characteristic before the exercise began and it remains so. Access is enforced in the database, not in the interface: **77 row-level security policies** across every table, with **52 triggers** holding the rules that policies cannot express.

- `role` and `status` are changeable only by the program office, enforced by a trigger rather than by hiding a form field.
- Avatar uploads are scoped per user by storage policy.
- The audit log records who changed a role, a status or a class, and no policy permits editing or deleting an entry — not even for an administrator. That is what makes it evidence.
- Service-role keys and the database URL appear in no frontend code and in no committed file.
- Every claim above is asserted by impersonating real users inside rolled-back transactions, so the tests prove the live rules rather than a copy of them.

Partial — two items were examined and deliberately left, and are listed as outstanding risks.

Maintainability

Modularity, reusability, analysability, modifiability and testability.

Where it is strong

379 assertions across 15 suites, all passing. They are not unit tests of helper functions; they sign in as real users inside a transaction, attempt the thing that should be refused, confirm it was, then attempt the paired thing that should be allowed and confirm that too. A rule that refused everything would fail them.

Where it was weak, and what it cost

The rebuild instructions were wrong. Several files redefine an object an earlier file owns, and whichever runs last wins. Three times, that was the older copy:

OBJECT	WINNER	WHAT WAS LOST
task_board_overview	dashboard.sql	ten columns, including <code>submitted_at</code> — the rebuilt database could not answer the application
guard_task_assignee	class-restore.sql	the locked-project and handed-in checks on claiming a task
guard_reassignment_request	reassignments.sql	the handed-in check

None of it showed in the live database, because the files had never been run in the documented order against it. Each object now has a single owner — the last file to define it, carrying every column and rule — and the order runs end to end, twice, followed by all fifteen suites.

Open, deliberately. There are still no JavaScript tests, no linter and no continuous integration. The four checkers this exercise produced — `schema-drift.mjs`, `contrast.mjs`, `ally-names.mjs` and the SQL suites — run only when someone remembers to run them. This was the one phase skipped, and it is the largest remaining gap in the model.

Flexibility

Adaptability, scalability, installability and replaceability.

Adaptability: the browser floor

The stylesheet compiles down to modern colour functions. Counted in the built CSS:

FEATURE	USES	FIRST SUPPORTED IN
@property	63	Chrome 85 · Safari 16.4 · Firefox 128
color-mix()	162	Chrome 111 · Safari 16.2 · Firefox 113
oklch()	22	Chrome 111 · Safari 15.4 · Firefox 113

The floor is **Chrome/Edge 111, Safari 16.4, Firefox 128**, now declared in `browserslist`. Below it the page does not degrade — those functions fail to parse rather than falling back, so every colour resolves to nothing and the page loads unreadable. A college lab on a pinned browser had no way of knowing why. The page now checks before booting and replaces itself with a plain sentence naming the versions needed.

Screens

VIEWPORT	PAGE	SIDEWAYS SCROLL	CONTENT PAST THE EDGE
360 × 740	landing	no	none
360 × 740	register, light	no	none
768 × 1024	landing	no	none
1024 × 800	landing	no	none
1440 × 900	landing	no	none

One real fault was found at 360 px and fixed: the landing navigation pushed the menu icon off the right edge, where it could not be tapped at all. The cause is worth recording — the button component sets a `display` utility in its own base class, and adding `hidden` alongside it does nothing, because competing display utilities are resolved by their order in the generated stylesheet, not by the order they are written.

Scalability

The quadratic query shapes described under Performance were the real scalability limit and are gone. What remains untested is behaviour at a full cohort's data volume; today's database holds 591 rows.

Partial. Adaptability is measured. Running a second program remains a code change rather than configuration — the school's identity is a constant — which is correct for the current scope and recorded here so it is a decision rather than an oversight.

Safety

Operational constraint, risk identification, fail-safe behaviour, hazard warning and safe integration — avoiding a state that puts people or their work at unacceptable risk.

For a coursework system the hazard is losing a term's work. Nineteen foreign keys cascade from the profiles table. Most of them should: a person's own claims, votes, messages and settings have no meaning without the person. Two did not delete the person's own rows — they deleted everybody else's.

KEY	WHAT ONE DELETE DESTROYED
<code>classes.professor_id</code>	the class, its group sets, every project, board, task, comment, file and enrolment — on live data that is 1 class, 3 projects, 26 tasks and 16 enrolments, with no undo and no warning
<code>projects.created_by</code>	the project and all of its groups' boards and work

Both are on `delete restrict` now. The delete is refused while the work exists, and the class must be handed over or archived first — which is what the accounts page had always said in words while the database quietly disagreed. There is no delete button for a professor in the product, but the hosting dashboard has one, and a constraint holds where a missing button does not.

A constraint that can never be satisfied is a trap rather than a safeguard, so the way out is tested too: hand the class to somebody else and the same delete succeeds. A second fault surfaced doing this — a trigger was pinning the old author back onto tasks, so the professor could not be deleted even after handing over. Fixed and asserted.

Alongside it: archiving is offered everywhere deleting is, every destructive dialog names what will be lost in counts rather than asking whether you are sure, and the audit log cannot be edited by anyone.

Addressed.

VERIFY

```
node scripts/db.mjs supabase/tests/safety.test.sql
```

The refusal, the survival of the work, the hand-over that succeeds, and the cascades that should still happen.

What is still outstanding

Nothing in this section is hidden in a footnote elsewhere. These are the things a panel should ask about, listed so they can be answered rather than discovered.

Decided against, by the owner

ITEM	RISK
The Anthropic API key has not been rotated	A key that has been exposed should be replaced regardless of whether misuse is suspected. Until it is, the AI task-drafting feature is the one credential outside the guarantees made under Security.
Email confirmation is off; outbound mail is not switched on	Accounts are usable without a verified address, and the system cannot notify anyone of anything by email. The interface no longer promises that it will.

Not done in this exercise

ITEM	CONSEQUENCE
No JavaScript tests, linter or continuous integration	Fifty-five pure functions across four modules — the ones computing what people read in analytics and reports — have no tests. The five checkers this exercise produced run only on request. This is the single largest gap against the model.
Safari, Firefox and a real Android device untested	One browser engine was available. It proves nothing about the other two. A matrix presented from a single engine would be a guess, so the rows are left blank in <code>docs/08-compatibility.md</code> with the exact steps to fill them.
No print preview of a real report	Reaching one needs a signed-in professor. The stylesheet is written and reviewed; it has not been put on paper.
Backup retention not confirmed	What the hosting plan actually retains, and whether point-in-time recovery is included, is a billing-tier property that has not been read from the dashboard. <code>docs/07-backup.md</code> holds a blank line for the answer.
No backup has ever been taken by hand	The client tools are not installed on the development machine, so no file size or duration can be quoted. A backup nobody has taken is a plan, not a backup.
Behaviour at cohort scale unmeasured	All timings are against 591 rows. The quadratic query shapes are fixed, which was the known limit, but the figures are small-data figures.

Reproducing every figure

Nothing in this report requires trusting it. Each command prints the numbers quoted above.

COMMAND	WHAT IT PROVES
<code>npm run build</code>	Type-checks and builds; the chunk sizes under Performance are its output.
<code>node scripts/contrast.mjs</code>	Every contrast ratio in both themes. Exits non-zero on a failure.
<code>node scripts/ally-names.mjs</code>	Icon-only buttons with no accessible name.
<code>node scripts/schema-drift.mjs</code>	Objects defined by more than one file, flagging any whose last definition shrank.
<code>node scripts/db.mjs</code> <code>supabase/tests/*.test.sql</code>	379 assertions against the live database, each rolled back.
<code>node scripts/resize-png.mjs</code> <code><file> <px></code>	The image downscaler, with its before and after byte counts.

Supporting documents in the repository

<code>docs/07-backup.md</code>	What is recoverable, the verified rebuild order, and what is not measured.
<code>docs/08-compatibility.md</code>	The browser floor, the tested viewports, and the matrix left for the owner to complete.
<code>supabase/tests/</code>	Fifteen suites. Every security and safety claim in this report resolves to one of them.

A closing note on honesty

Two claims in earlier drafts of this work were wrong and are corrected here: the stylesheet was said to have no focus styles when a global rule already existed, and an early version of the accessible-name checker reported fourteen failures that were all false. Both were found by checking rather than by remembering. A quality report that never corrects itself has not been checked either.